Oráculo Tainacan

Busca em linguagem natural e assistente de conversa (BIA) para acervos Tainacan.

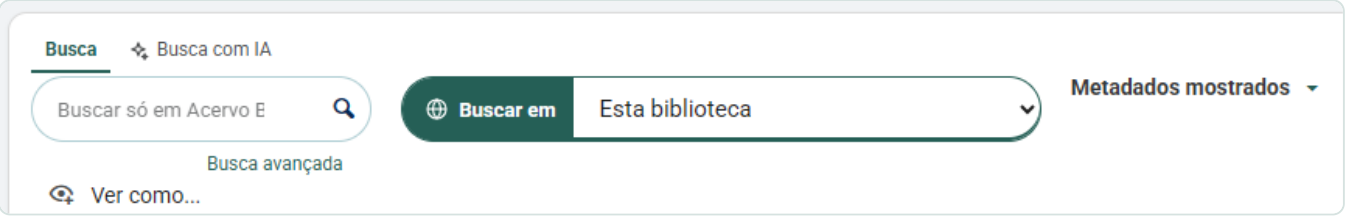
Versão 2.13.0 · outubro de 2026 · repositório `marcossigismundo/oraculo_tainacan`

O Oráculo deixa o visitante perguntar ao acervo do jeito que perguntaria a uma pessoa, como “livros sobre a seca no sertão”. Ele devolve os itens mais próximos **pelo sentido**, e não só pelas palavras, e escreve uma síntese que cita esses itens. Tudo responde a partir do que está catalogado: o modelo de linguagem só lê os itens recuperados.

Busca com IA
Uma aba ao lado da busca do Tainacan. Mostra os itens relacionados em cartões e uma resposta com citações numeradas [1], [2]…

BIA
Bibliotecária de IA em janela de conversa. Lembra o que foi perguntado antes e mostra as fontes de cada resposta.

Conteúdo dos documentos
Encontra itens pelo texto dos PDFs (camada de texto ou OCR) e aponta arquivo e página.



Na listagem de itens, a aba “Busca com IA” fica ao lado da busca tradicional do Tainacan, que continua funcionando igual.

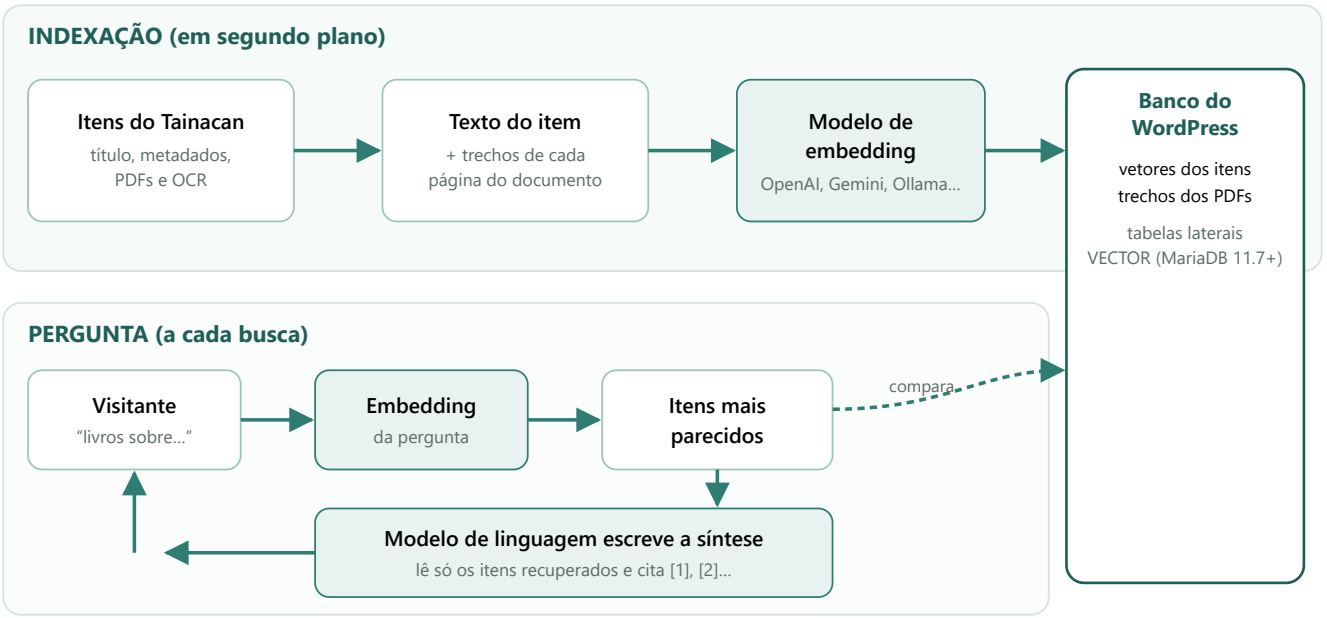
Sumário

1. Como funciona, em uma figura
2. O caminho de uma pergunta
3. A BIA
4. Busca dentro dos documentos
5. Indexação: como o acervo entra
6. Busca vetorial no banco (MariaDB 11.7+)
7. Configuração
8. Operação no dia a dia
9. Problemas comuns
10. Limites, custos e privacidade
11. Uso intenso: muita gente ao mesmo tempo

> **Em uma frase:** o plugin transforma cada item em um *vetor*, uma lista de números que representa o sentido do texto. Depois compara o vetor da pergunta com os do acervo para achar os mais parecidos, e só então pede ao modelo de linguagem um texto sobre esses itens.

1. Como funciona, em uma figura

São dois momentos independentes: a **indexação**, que prepara o acervo uma vez e depois acompanha as mudanças, e a **pergunta**, que acontece a cada busca.

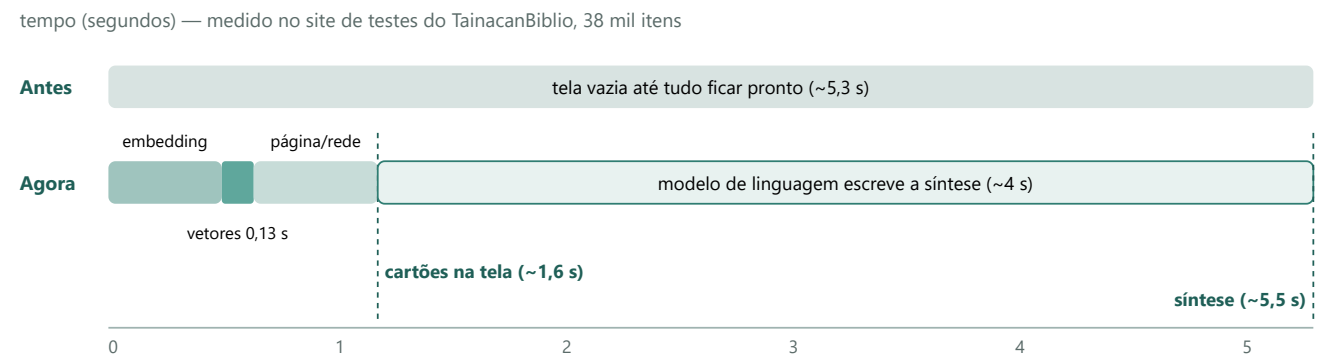


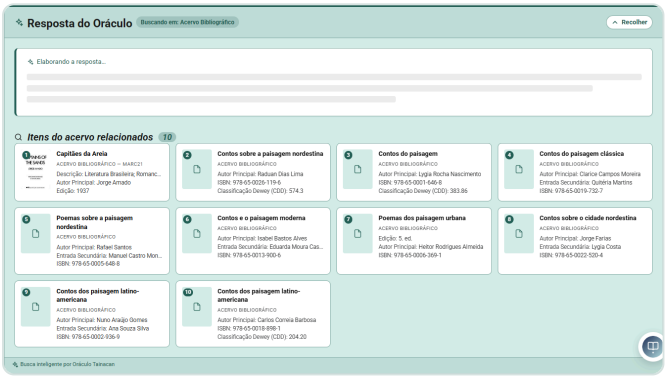
Peça	O que faz	Onde se configura
Modelo de embedding	Transforma texto em vetor. O mesmo modelo serve o acervo e a pergunta: trocar de modelo exige reindexar.	Configurações → Provedores
Banco vetorial	O próprio banco do WordPress. No MariaDB 11.7+, a comparação acontece dentro do banco (seção 6).	Automático
Modelo de linguagem	Escreve a resposta da busca e as falas da BIA a partir dos itens encontrados.	Configurações → Provedores
Fila de indexação	Acompanha itens criados, editados e removidos e atualiza os vetores sozinha.	Indexação

Provedores aceitos: OpenAI, Google Gemini, Claude (Anthropic), DeepSeek, Groq e Ollama, que roda no próprio servidor, sem enviar dados para fora. Há ainda o CLIP, da API de IA do IBRAM, para busca visual por imagem.

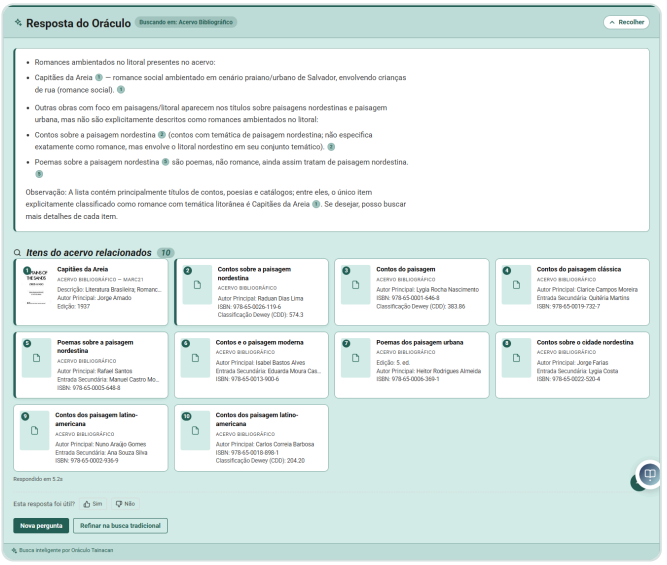
2. O caminho de uma pergunta

Desde a versão 2.12 a busca acontece **em duas etapas**. O modelo de linguagem responde por cerca de 85% do tempo total, então os cartões dos itens aparecem antes, e a síntese entra no mesmo painel quando fica pronta.





Etapa 1: cartões já na tela; a síntese aparece como “Elaborando a resposta…”.



Etapa 2: a síntese entra no lugar, com as citações ligadas aos cartões.

Passo a passo

1. **Pergunta.** O plugin reduz a frase ao assunto (“o que tem sobre o nordeste?” → “nordeste”) e calcula o embedding. Se a mesma pergunta foi feita nas últimas 2 horas, reaproveita o embedding.
2. **Recuperação.** Compara o vetor da pergunta com os do acervo, no escopo da tela (a coleção aberta ou o acervo todo), e junta os trechos de PDF que batem. Devolve os cartões e um *bilhete* de uso único.
3. **Síntese.** Com o bilhete, o modelo de linguagem recebe os itens já recuperados, sem refazer a busca, e escreve uma resposta curta que cita os cartões.
4. **Cache.** A resposta completa fica guardada por 1 hora. A mesma pergunta volta em uma chamada só, em menos de 1 segundo.

> Clicar numa citação [n] leva ao cartão correspondente. “Refinar na busca tradicional” leva a pergunta para a busca nativa do Tainacan, com facetas e filtros.

3. A BIA

A BIA é a bibliotecária de IA. Aparece como um botão no canto da página, ou dentro de uma página pelo shortcode `[oraculo_chat]`. A diferença para a busca é a **conversa**:

- **Memória:** lembra as perguntas anteriores. Uma pergunta como "e quem escreveu?" herda o assunto da conversa.
- **Mesmo escopo e mesma recuperação da busca:** a BIA vê os mesmos itens que a aba "Busca com IA" mostraria para a mesma pergunta.
- **Fontes:** toda resposta lista os itens usados, com link. Quando o item vem de um PDF, mostra o arquivo e a página.
- **Avaliação:** 👍 / 👎 em cada resposta alimenta o painel de Analytics.



A BIA respondendo, com as fontes do acervo.

Shortcodes

Shortcode	Uso
`[oraculo_search]`	Caixa de busca com IA em qualquer página
`[oraculo_chat height="600px" collections="12,15"]`	BIA embutida, com altura e coleções definidas

> A BIA liga e desliga em Configurações → Geral; a posição da janela fica em Aparência. A aba "Busca com IA" só carrega nas listagens de itens do Tainacan.

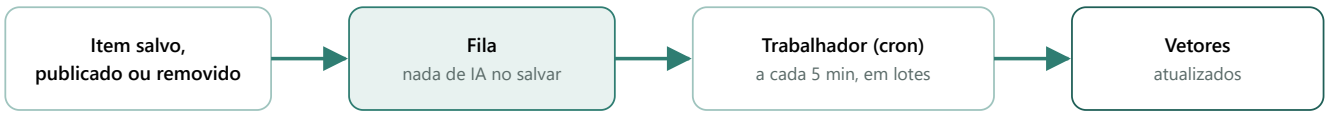
4. Busca dentro dos documentos

Quando o campo **document** está marcado em Configurações → Avançado → Campos para indexar, o plugin lê o texto dos anexos de cada item e o divide em trechos de cerca de 1.600 caracteres, um conjunto por página, cada um com o seu vetor. O texto vem de três fontes, nesta ordem:

1. o OCR do plugin **Tainacan OCR Search**, para documentos digitalizados;
2. a camada de texto dos PDFs, extraída com `pdftotext`;
3. o índice de conteúdo que o próprio Tainacan guarda.

Na busca, o item ganha pontos pelo melhor trecho. O cartão mostra "No documento", com o arquivo, a página e o trecho, e o link abre o PDF já naquela página. O Oráculo **não faz OCR**: ele só consome o OCR do Tainacan OCR Search.

5. Indexação: como o acervo entra



Uma reconciliação diária confere se algum item ficou para trás. Item que não mudou (mesmo conteúdo) não é reprocessado.

- **Primeira indexação de uma coleção:** Oráculo → Indexação → escolher a coleção. Em acervos grandes, prefira o WP-CLI: `wp oraculo index --collection=<id>`.
- **O que entra no texto do item:** os campos marcados em Configurações → Avançado → Campos para indexar (título, descrição, metadados e, opcionalmente, o documento).
- **Só itens publicados** entram na busca pública. Despublicar ou excluir um item tira o vetor e os trechos dele.
- **Trocou o modelo de embedding?** Reindexe tudo. Vetores de modelos diferentes não se comparam.

6. Busca vetorial no banco (MariaDB 11.7+)

Até a versão 2.10, o PHP lia todos os vetores e fazia a comparação um a um. Isso limitava a busca a acervos pequenos: com 10 mil itens, eram 6,5 s e ~500 MB de memória por busca. A partir da 2.11, se o banco for **MariaDB 11.7 ou mais novo**, o plugin mantém tabelas laterais com o tipo `VECTOR`. A comparação acontece **dentro do banco**, e o PHP recebe apenas os ids dos mais próximos.

Escopo da busca (38.102 vetores reais)	Antes (PHP)	Agora (no banco)
Acervo todo	512 ms	**115 ms**
Uma coleção com 30 mil itens	419 ms	**135 ms**
Uma coleção com 8 mil itens	233 ms	**33 ms**

- **Resultado exato:** cada vetor é comparado, sem aproximação. O método antigo perdia parte dos vizinhos (82% em comum no top-20 da coleção menor).
- **Automático:** as tabelas são criadas e preenchidas em segundo plano (38 mil vetores em ~45 s) e acompanham toda gravação e remoção.
- **Sem MariaDB 11.7+** (MySQL ou MariaDB anterior) nada muda: a busca segue no método antigo.

> **Atenção ao copiar o banco** para um servidor com MySQL ou MariaDB anterior ao 11.7: rode antes `wp oraculo native drop`. Esses servidores não restauram as tabelas com tipo `VECTOR`. No servidor novo, a busca volta ao método antigo sem nenhum ajuste.

7. Configuração

Fica em **Tainacan** → **Oráculo**. As telas são: Painel, Indexação, Configurações, **Diagnóstico** (seção 11), Analytics e Debug. As abas de Configurações:

Aba	O que se ajusta
Provedores	Provedor e chave de API, modelo de linguagem e modelo de embedding; teste de conexão.
Geral	Ligar a busca e a BIA; quantos itens mostrar (padrão 10); similaridade mínima (padrão 0,3); limite de tokens da resposta.
Prompts	Instruções de sistema, de busca e de conversa, globais ou por coleção. As regras de citação [n] são acrescentadas pelo plugin.
Aparência	Cores e textos da BIA, posição da janela, mostrar fontes e relevância, feedback.
Tainacan	Aba "Busca com IA" nas listagens: rótulo, texto do campo, escopo (coleção aberta ou acervo todo), sugestões de perguntas e link direto (`?oraculo_q=`).
Avançado	**Campos para indexar** (incluindo o documento), tamanho do lote, tempo limite das requisições, duração do cache (padrão 1 h), modo debug e manutenção (limpar cache, limpar vetores, exportar configurações).

8. Operação no dia a dia

Todos os comandos rodam a partir da pasta do site, com `wp oraculo …`.

Comando	Para quê
`status`	Situação geral: provedores, vetores, fila.
`index --collection=ID`	Indexa ou atualiza uma coleção.
`documents index|status --collection=ID`	Trechos dos PDFs: indexar ou ver o que falta (inclui PDFs que precisam de OCR).
`queue`	Fila da indexação automática.
`native status|sync|rebuild|drop`	Busca no banco: situação, preencher agora, refazer do zero, remover (antes de levar o banco para outro servidor).
`search "pergunta" --limit=5`	Testa uma busca completa pelo terminal, com tempo e fontes.
`test_connection` · `providers`	Confere chaves e modelos dos provedores.
`clear_vectors [--collection=ID]`	Apaga vetores, de uma coleção ou de todas. Depois, reindexe.
`stats` · `export` · `optimize`	Estatísticas de uso, exportação e manutenção das tabelas.

9. Problemas comuns

Sintoma	Causa provável	O que fazer
Nenhum resultado para nada	Coleção não indexada, ou modelo de embedding trocado sem reindexar	`wp oraculo status`; reindexar a coleção
Item novo não aparece	A fila ainda não rodou (o cron roda a cada 5 min)	`wp oraculo queue`; conferir se o WP-Cron roda no servidor
Busca lenta (vários segundos antes dos cartões)	Busca no banco desligada ou ainda preenchendo	`wp oraculo native status`; se preciso, `native sync`
Cartões rápidos, síntese demorada	Normal: é o modelo de linguagem (~2 s com gpt-4.1-mini, ~6 s com gpt-5-nano)	Modelo recomendado em Provedores; menos itens em Geral (contexto menor)
"Muitas requisições"	Limite por visitante (10 buscas/min) ou teto do site (300 chamadas de IA/min; 60 em quem atualizou)	Esperar um minuto; ajustar em **Diagnóstico → Limites de uso**, ou liberar a rede da escola/museu
Site inteiro lento quando muita gente usa a IA	Busca e páginas disputam os mesmos processos PHP	Seção 11: modo rápido e processos PHP próprios
PDF não é encontrado pelo conteúdo	PDF digitalizado sem OCR, ou campo "document" desligado	`wp oraculo documents status --collection=ID`; rodar o OCR
Resposta "estranha"	A IA só fala do que achou: o problema costuma estar nos dados	Ver os cartões: se não são pertinentes, rever metadados e campos indexados

10. Limites, custos e privacidade

- **Limites de uso:** 10 buscas e 20 mensagens da BIA por minuto por visitante, e 300 chamadas de IA por minuto no site todo (60 em quem atualizou de versão anterior), ajustáveis na aba Diagnóstico. A busca em duas etapas conta uma vez só. Redes liberadas (escola, museu atrás de um único IP) ficam sem o limite por visitante.
- **Custo por busca** (OpenAI): um embedding da pergunta e cerca de 2 mil tokens no modelo de linguagem. Pela tabela da OpenAI, cerca de US$ 1 a cada mil perguntas com gpt-4.1-mini e US$ 0,30 com gpt-5-nano (confira na sua conta). O cache de 1 hora evita pagar de novo pela mesma pergunta, inclusive na primeira mensagem da BIA.
- **O que sai do servidor:** com provedor externo, a pergunta e o texto dos itens recuperados vão ao provedor. Com **Ollama**, nada sai do servidor, mas a resposta fica bem mais lenta numa máquina sem GPU.
- **Sugestões públicas** só mostram buscas feitas por pelo menos 3 visitantes diferentes. O que uma pessoa digitou sozinha nunca vira sugestão.
- **Registros:** buscas e conversas ficam em tabelas próprias e são apagadas por uma rotina diária de retenção.

Medições feitas em 29/10/2026 no site de testes do TainacanBiblio (VPS de 1 vCPU e 4 GB, MariaDB 11.8, 38.102 vetores de 1.536 dimensões, embeddings `text-embedding-3-small`, respostas gpt-5-nano e gpt-4.1-mini). Detalhes técnicos em `docs/ANALISE-LIMITE-VETORES.md` e `docs/CONTEXT.md` no repositório.

11. Uso intenso: muita gente ao mesmo tempo

Cada pergunta prende um processo PHP do servidor enquanto espera o provedor de IA responder. Com muita gente ao mesmo tempo, esses processos acabam, as perguntas entram em fila e o resto do site fica lento junto. Para medir, disparamos **40 buscas e 30 conversas com a BIA** contra o site de testes (VPS de **1 núcleo** e 4 GB) e repetimos com cada ajuste.

Todos no mesmo segundo (o pior caso)

Configuração	Tudo atendido	Cartões	Busca completa	BIA	Página inicial
Antes (6 processos, gpt-5-nano, todos os plugins)	64 s	10,6 s	48 s	27 s	22,6 s
+ modelo gpt-4.1-mini	40 s	7,8 s	32 s	18,5 s	10,7 s
+ processos PHP próprios	38 s	10,5 s	31 s	22 s	**0,8 s**
+ **modo rápido** (a configuração recomendada)	**26,5 s**	**4,9 s**	**22 s**	**14 s**	**0,7 s**

Tempos por pessoa: mediana (metade esperou menos). Nenhum dos 770 pedidos dos testes falhou.

As mesmas 70 pessoas chegando ao longo de 20 segundos

Configuração	Cartões	Busca completa	BIA	Página inicial
Antes	6,8 s	19,4 s	9,6 s	5,2 s
Recomendada	**0,9 s**	**3,0 s**	**2,2 s**	**0,8 s**

Espalhadas por um minuto inteiro, as duas configurações atendem como se houvesse uma pessoa só (cartões em 0,6–0,7 s). A diferença aparece nos picos.

O que cada ajuste faz

- **Modelo de linguagem.** gpt-4.1-mini respondeu em ~1,9 s, com texto curto e correto; gpt-5-nano em ~6 s (ele "raciocina" antes); gpt-4o-mini inventou links. É o padrão nas instalações novas.
- **Modo rápido** (plugin, um clique). Nas rotas da busca e da BIA, carrega só o Oráculo: 0,05 s de CPU por pedido, contra 0,13 s com todos os plugins. Resultados idênticos.
- **Processos PHP próprios** (servidor). Não aceleram o Oráculo; impedem que ele derrube o site durante o pico.
- **Limites de uso.** O teto antigo (60/min) recusaria parte desse pico; nas instalações novas o padrão é 300. Quem atualiza mantém 60 até mudar na aba Diagnóstico.
- **Mais processos não ajudam** num núcleo só: com 16, a CPU saturou e os cartões pioraram. Acima disso, o que rende é CPU (2 a 4 núcleos).

Modelo local (Ollama)

Na mesma VPS, sem placa de vídeo: **qwen2.5 1.5B** levou 58–70 s e afirmou que dois livros tratavam de Brasília (não tratavam); **qwen3-vl 2B**, 210 s, com links inventados. O Ollama atende uma pergunta por vez: 70 pessoas esperariam mais de uma hora. Serve para tarefas em segundo plano ou num servidor com GPU, não para a busca ao vivo.

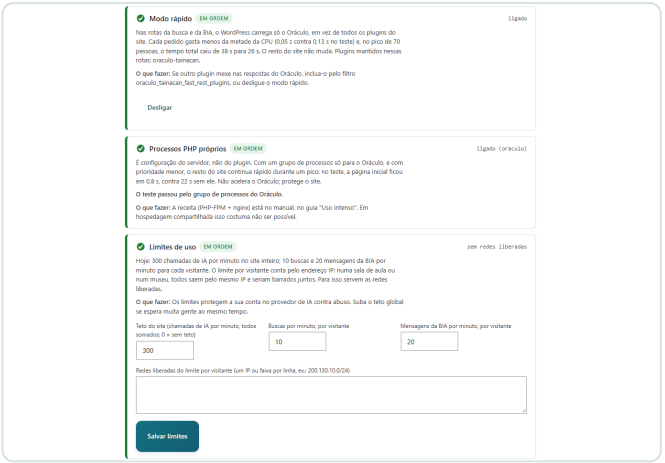
> **O que vem com o plugin e o que é do servidor:** modelo, modo rápido, limites e cache vêm com o plugin e funcionam em qualquer hospedagem (o modo rápido precisa poder gravar em `wp-content/mu-plugins`). Os processos PHP próprios são configuração do servidor.

A aba Diagnóstico

Em **Tainacan → Oráculo → Diagnóstico**, cada item mostra a situação (verde, amarelo, vermelho), o que ela significa e o que fazer. O botão **Testar agora** faz uma busca de teste pelo mesmo caminho do visitante e mede cada etapa; é ele que confirma o modo rápido e os processos próprios funcionando de verdade.



Placar, teste ao vivo e as primeiras verificações. Carga do WordPress em 0,04 s com o modo rápido.



Modo rápido (liga e desliga com um clique), processos próprios confirmados pelo teste e o formulário dos limites.

Receita: processos PHP próprios (PHP-FPM + nginx)

Para quem administra o servidor. Em hospedagem compartilhada isso costuma não ser possível; o Oráculo funciona igual, só sem a proteção do resto do site.

1. Um grupo de processos novo, cópia do grupo do site (mesmo usuário), em `/etc/php/8.2/fpm/pool.d/zz-meusite-oraculo.conf`:

```
[meusite-oraculo]
listen = 127.0.0.1:17101          ; porta livre
user = meusite
group = meusite
pm = ondemand
pm.max_children = 10              ; num núcleo só, mais que 10 não ajuda
process.priority = 10             ; as páginas do site passam na frente
env[ORACULO_PHP_POOL] = oraculo   ; é por ela que o Diagnóstico reconhece o grupo
```

2. No nginx, no nível `http` (um arquivo em `sites-enabled` serve), escolha o grupo pelo endereço:

```
map $request_uri $meusite_php {
  ~^/wp-json/oraculo/  127.0.0.1:17101;   # site em subpasta: ~^/subpasta/wp-json/oraculo/
  default              127.0.0.1:17001;   # o grupo de sempre do site
}
```

e, no bloco `location ~ \.php$` do site, troque o `fastcgi_pass` fixo por `fastcgi_pass $meusite_php;`.

3. `php-fpm8.2 -t && nginx -t`, recarregue os dois e clique em **Testar agora**: "Processos PHP próprios: sim (oraculo)".

> **Painéis como o CloudPanel** regravam o vhost quando ele é editado pela interface, e a linha do `fastcgi_pass` volta ao original. Depois de mexer no site pelo painel, rode o teste de novo.